

The lock-free SPSC ring (feed -> book handoff)

Learning notes for `include/lob/spsc_ring.hpp`. Two threads: a **feed** thread parses market data, a **book** thread applies it. We need to pass messages from one to the other millions of times a second without a lock. This explains how a single-producer/single-consumer (SPSC) lock-free ring does it, and the memory-ordering that makes it correct.

1. Why not just a mutex + queue?

A `std::mutex`-guarded `std::queue` works, but on a hot path it hurts:

- **Contention:** producer and consumer fight over the same lock; each `lock / unlock` is an atomic RMW plus, under contention, a kernel wait.
- **Latency spikes:** if the consumer holds the lock and gets descheduled, the producer stalls (a form of priority inversion). Tail latencies blow up - exactly what HFT cares about.
- **Allocation:** `std::queue<T>` allocates nodes.

A lock-free ring removes all three: no lock, no blocking, no allocation after construction.

2. The SPSC simplification

The general multi-producer/multi-consumer case needs compare-and-swap (CAS) loops. **SPSC is special:** exactly one thread pushes, exactly one pops. So:

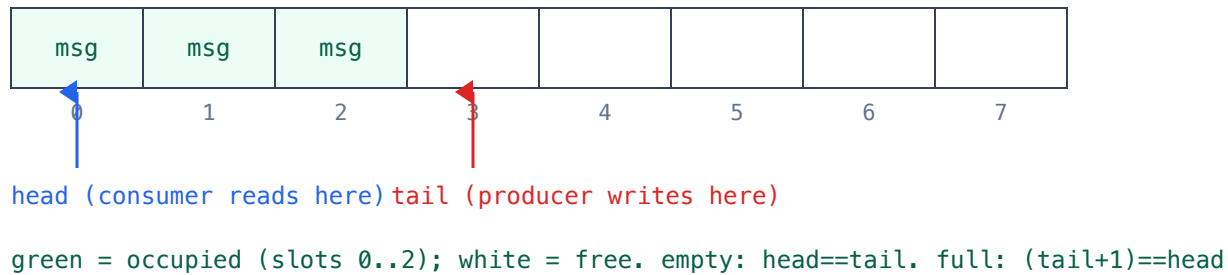
- the **producer owns** `tail_` (only it writes tail),
- the **consumer owns** `head_` (only it writes head).

Neither writes the other's cursor, so there is no write-write race - no CAS needed, just plain loads/stores plus the right memory fences. That is why SPSC is so cheap.

3. The ring

A fixed array used circularly. `head_` is where the consumer reads next; `tail_` is where the producer writes next. Both advance modulo the capacity (a power of two, so `& (cap-1)` replaces modulo). One slot is left empty so we can tell **full** from **empty**.

Ring buffer: producer writes at tail, consumer reads at head



4. The crux: the acquire/release handshake

The danger: the consumer must **never read a slot before the producer's write to that slot is visible**. A plain `bool ready` flag is not enough - the compiler or CPU can reorder the slot write past the flag write. We use **release/acquire** ordering to forbid that reordering.

```
bool push(const T& v) {                                // producer
    const std::size_t t = tail_.load(relaxed);
    const std::size_t next = (t + 1) & mask_;
    if (next == head_.load(acquire)) return false;    // full? (observe consumer)
    buf_[t] = v;                                     // (1) write the slot
    tail_.store(next, release);                       // (2) publish it
    return true;
}

bool pop(T& out) {                                     // consumer
    const std::size_t h = head_.load(relaxed);
    if (h == tail_.load(acquire)) return false;      // (3) observe producer
    out = buf_[h];                                    // (4) read the slot
    head_.store((h + 1) & mask_, release);           // publish free space
    return true;
}
```

The pairing that makes it safe:

- `tail_.store(release)` at **(2)** means *"everything I wrote before this store (the slot at (1)) is visible to any thread that reads this value with acquire."*
- `tail_.load(acquire)` at **(3)** means *"once I see the new tail, I also see the writes that happened-before the release that produced it."*

So (1) happens-before (4): the slot data is guaranteed visible before the consumer reads it.

`head_` release/acquire does the mirror image for free space.

happens-before: the slot write is visible before the consumer reads it

PRODUCER

CONSUMER

(1) buf[tail] = msg

(2) tail.store(next, RELEASE)



(3) tail.load(ACQUIRE)

(4) read buf[head]

RELEASE (2) sync-with ACQUIRE (3), so (1) happens-before (4)

5. False sharing: pad the cursors

`head_` and `tail_` are written by different threads. If they share a cache line, every update by one core **invalidates the line in the other core's cache** - they ping-pong a 64-byte line back and forth (false sharing), and your "lock-free" ring crawls. Fix: put each on its own line.

```
alignas(64) std::atomic<std::size_t> head_{0}; // consumer's cache line
alignas(64) std::atomic<std::size_t> tail_{0}; // producer's cache line
```

6. Full / empty, without a size counter

A shared `size` counter would be another contended atomic. Instead we derive both states from the two cursors, sacrificing one slot:

- **empty:** `head == tail`
- **full:** `(tail + 1) & mask == head`

The producer only reads `head` (acquire) to check full; the consumer only reads `tail` (acquire) to check empty. Each still owns its own cursor.

7. Latency vs throughput (an honest-measurement lesson)

When I first benchmarked this, "latency" came out at ~6.6 **ms** - absurd for a ring. The bug was in the *measurement*: the producer flooded the 64k-slot ring, so every message sat behind a full backlog. That is **queue residency**, not handoff latency - and by **Little's law** (`latency = queue_depth / throughput`) a deep queue *must* show high latency.

So I report two numbers that trade off against each other:

- **handoff latency (1 message in flight):** p50 **125 ns**, p99 167 ns, p99.9 250 ns - the true time for one message to cross the ring and be applied.
- **sustained throughput (flood):** **~8.2 M msg/s** cross-thread.

Lesson: a single "latency" number from a saturated queue is meaningless; state the load.

8. Trade-offs / next

- **SPSC only.** More than one producer or consumer needs CAS (MPSC/MPMC) and is slower.
- **Bounded.** A full ring makes the producer spin or drop - you must size it and decide policy.
- **Batching** amortizes the release-store across several messages for more throughput.
- On x86 Linux: pin the two threads to isolated cores (`taskset -c`) and use `rdtsc` - the cross-core cache-line handoff is what the 125 ns mostly is, and a tuned box tightens the tail.